Surname: | **First name:**

UO:

Practical Exam

29 October 2026

Implement `ReadOnlySortedCollection<E>` by extending `AbstractCollection<E>`. Use singly linked nodes and keep the elements in ascending order. The collection must not contain `null` elements. Duplicates are allowed. Do not store the elements in an array or another collection.

Task 1. Representation and sorted order [4 points]

Define the node class and implement the methods needed to make the collection fully functional. Once constructed, the collection must be unmodifiable.

The supplied class skeleton contains the comparison method `compare(E first, E second)`, as used in Session 4. It uses the instance field `comparator`. Complete the rest of the class.

Elements must be ordered using either natural order or a comparator supplied to the constructor. Elements that compare as equal must retain their original order in the source collection. All duplicates must be kept.

Task 2. Constructors [3 points]

Provide the following constructors:

- `ReadOnlySortedCollection()`: creates an empty collection that uses natural order.
- `ReadOnlySortedCollection(Comparator<? super E> comparator)`: creates an empty collection that uses the specified comparator.
- `ReadOnlySortedCollection(Collection<? extends E> source)`: creates a collection containing the elements of `source`, in natural order.
- `ReadOnlySortedCollection(Collection<? extends E> source, Comparator<? super E> comparator)`: creates a collection containing the elements of `source`, ordered using the specified comparator.

A `null` comparator selects natural order. A `null` source or a `null` element must cause `NullPointerException`. With natural order, an element that is not comparable or cannot be compared with the other elements must cause `ClassCastException`, even if it is the first element.

Task 3. Iterator [2 points]

Write a private inner iterator class and return a new instance from `iterator()`. Iterators must work independently and return the elements in sorted order. When no elements remain, `next()` must throw `NoSuchElementException`. The iterator must not support removal.

Operations that would modify the collection must throw `UnsupportedOperationException`. Calls that make no change need not throw.

Task 4. Main program and supplied tests [1 point]

Write `app.App` to carry out the following checks:

- Create empty collections and construct collections from other collections.
- Check natural order, reverse order and duplicates. Use objects with a priority and a name to check that objects with the same priority keep their original order.
- Change the source collection and check that the new collection is unaffected.
- Check iteration, an exhausted iterator, `null` elements, a non-comparable element and attempts to modify the collection.

Print the expected and actual results. Catch expected exceptions so the remaining checks can run.

The project includes JUnit tests and the required library. Run the supplied tests; do not modify them. They check constructors, ordering, the order of duplicates, independent copies, exhausted iterators and unsupported operations. All supplied tests must pass. The main program is worth one point; the collection is assessed in Tasks 1–3. Passing the tests does not replace the main program or the other requirements in this handout. No additional public methods are required.